This technical report provides the detailed pseudocode of the proposed greedy algorithms and the proofs of the theoretical results. Algorithm 1 (Followers) describes the procedure for computing the follower set induced by an anchored edge. Algorithm 2 (GIA) presents the greedy incremental anchoring strategy, which iteratively selects the edge–weight increment pair with the largest follower gain per unit budget. Algorithm 3 (GPA) accelerates GIA by using candidate reduction, upper-bound pruning, and intermediate-result reuse, while preserving the same greedy selection criterion.

A Pseudocode of Followers

Algorithm 1: Followers(G, e, δ_e, s, C_s)

$w(e) \leftarrow w(e) + \delta_e$;
$\sigma(x) \leftarrow \mathsf{U}$ for all $x \notin C_s$;
$\mathsf{PQ} \leftarrow$ empty min-priority queue keyed by coreness–layer pair;
for *each* $x \in \{u, v\}$ ***with*** $x \notin C_s$ **do**
  PQ.push(x); $\sigma(x) \leftarrow \mathsf{Q}$;
while $\mathsf{PQ} \neq \emptyset$ **do**
  $x \leftarrow \mathsf{PQ}$.pop$()$;
  $w^+_\sigma(x) \leftarrow \sum_{y \in N^+_\sigma(x)} w(x, y)$;
  if $w^+_\sigma(x) \geq s$ **then**
    $\sigma(x) \leftarrow \mathsf{R}$;
    for *each* $y \in N(x, G)$ ***with*** $y \notin C_s$, $x \prec y$, $\sigma(y) = \mathsf{U}$ **do**
      PQ.push(y); $\sigma(y) \leftarrow \mathsf{Q}$;
  else
    $\sigma(x) \leftarrow \mathsf{D}$;
    if $x \in \{u, v\}$ **then** $w(e) \leftarrow w(e) - \delta_e$; **return** $\emptyset$;
    $\mathsf{Q} \leftarrow$ queue$()$; Q.push(x);
    while $\mathsf{Q} \neq \emptyset$ **do**
      $y \leftarrow \mathsf{Q}$.pop$()$;
      for *each* $z \in N(y, G)$ ***with*** $\sigma(z) = \mathsf{R}$ **do**
        $w^+_\sigma(z) \leftarrow w^+_\sigma(z) - w(y, z)$;
        if $w^+_\sigma(z) < s$ **then**
          $\sigma(z) \leftarrow \mathsf{D}$;
          if $z \in \{u, v\}$ **then** $w(e) \leftarrow w(e) - \delta_e$; **return** $\emptyset$;
          Q.push(z);
$w(e) \leftarrow w(e) - \delta_e$;
return $\{x : \sigma(x) = \mathsf{R}\}$

Algorithm description. Algorithm 1 computes the follower set induced by an anchored edge via a localised simulation of the s-core peeling process. Each non-core node carries a state $\sigma(\cdot) \in \{\mathsf{U}, \mathsf{Q}, \mathsf{R}, \mathsf{D}\}$ (unseen, queued, retained, discarded), and the support neighbours $N^+_\sigma(x)$ are those of Definition 8 in the main paper: core neighbours, non-discarded neighbours that succeed x in the coreness–layer order, and queued or retained neighbours. The procedure maintains three invariants: a node is pushed into PQ at most once (only while U), a node is discarded at most once (a discarded node is never reinserted), and the weight of an edge (y, z) is subtracted from $w^+_\sigma(z)$ at most once, namely when y is discarded. If either endpoint is discarded, the computation stops and returns $\emptyset$. Starting from the anchored endpoints, the algorithm explores only nodes that are reachable through neighbours with higher coreness–layer pairs, following the upstair paths determined by the onion-layer structure. A node is retained as a follower if its support weighted degree meets the s-core threshold; otherwise, it is discarded. Discarding a node may reduce the support of neighbouring retained followers, potentially triggering further removals. This cascading update exactly mirrors the behaviour of the s-core decomposition, but is restricted to the structurally affected region. The process terminates when no further admissible nodes remain, and the retained follower nodes constitute the final follower set.

B Pseudocode of GIA

Algorithm 2: GIA(G, b, s)

$G' \leftarrow G$, $A \leftarrow \emptyset$, $sum \leftarrow 0$;
$C_s \leftarrow s$-core(G');
while $sum < b$ **do**
  $E^* \leftarrow \left\{(u, v) \in \binom{V}{2} \mid \{u, v\} \not\subseteq C_s\right\}$;
  $e^* \leftarrow none$, $\delta^* \leftarrow 0$, $max_FR \leftarrow 0$;
  for *each* $e = (u, v) \in E^*$ **do**
    $\tilde{\delta}_e \leftarrow s - \min\{c(u), c(v)\}$;
    if $sum + \tilde{\delta}_e \leq b$ **then**
      $\overline{\delta}_e \leftarrow s - w_{G'}(e)$;
      $\delta_{\text{test}} \leftarrow \min\{\overline{\delta}_e, b - sum\}$;
      $\mathcal{F}(e, \delta_{\text{test}}) \leftarrow$ Followers$(G', e, \delta_{\text{test}}, s, C_s)$;
      if $\mathcal{F}(e, \delta_{\text{test}}) \neq \emptyset$ **then**
        $H \leftarrow C_s \cup \mathcal{F}(e, \delta_{\text{test}})$;
        $\delta_e \leftarrow \max\{s - w(u, G'[H]), s - w(v, G'[H])\}$;
        $FR(e) \leftarrow |\mathcal{F}(e, \delta_{\text{test}})|/\delta_e$;
        if $FR(e) > max_FR$ ***or*** $(FR(e) = max_FR$ ***and*** $e \prec_{\text{tb}} e^*)$ **then**
          $e^* \leftarrow e$, $\delta^* \leftarrow \delta_e$;
          $max_FR \leftarrow FR(e)$;
  if $e^* = none$ **then**
    break;
  $w_{G'}(e^*) \leftarrow w_{G'}(e^*) + \delta^*$;
  $sum \leftarrow sum + \delta^*$;
  $A \leftarrow A \cup \{\langle e^*, \delta^* \rangle\}$;
  $C_s \leftarrow s$-core(G');
return A

Algorithm description. Algorithm 2 presents the baseline greedy incremental anchoring procedure of GIA. Throughout Algorithms 2 and 3, follower ratios are compared exactly (by cross-multiplication of integers), and ties are broken by the fixed lexicographic order $\prec_{\text{tb}}$ on canonical pairs $(\min\{u, v\}, \max\{u, v\})$; this is the tie-breaking order assumed in Property 1. Starting from the initial s-core, the algorithm iteratively evaluates candidate edges whose endpoints are not both contained in the current s-core. For each candidate edge, GIA first checks budget feasibility using the lower-bound increment $\tilde{\delta}_e$ and performs a single follower computation with $\delta_{\text{test}} = \min\{\overline{\delta}_e, b - sum\}$. If followers are found, it computes the MEI δ_e from the resulting follower set and evaluates the corresponding follower ratio. At each iteration, the edge achieving the largest follower ratio is selected, its weight is increased by δ_e, and the s-core is updated. This process repeats until the budget is exhausted or no feasible positive-gain candidate remains, yielding a greedy baseline for comparison.

C Pseudocode of GPA

Algorithm description. Algorithm 3 presents the pseudocode of GPA, an accelerated variant of GIA that reduces the candidate space while preserving the same greedy selection criterion. GPA exploits the locality of follower propagation by organising nodes outside the s-core into connected components and maintaining, for each component, the best candidate edge–weight increment pair identified so

Algorithm 3: GPA(G, b, s)

```
1  G' ← G, A ← ∅, sum ← 0;
2  C_s ← s-core(G');
3  M ← buildComponents(G',b,s);
4  while sum < b do
5      L ← list();
6      e* ← none, δ* ← 0, max_FR ← 0;
7      while M ≠ ∅ do
8          (e*, δ*, max_FR) ← arg max_{(e,δ_e,FR)∈M} FR; // ties by ≺_tb
9          if sum + δ* ≤ b then break;
10         c ← the component of e*; remove the entry of c from M;
11         M[c] ← searchComponent(G', c, b − sum);
12         e* ← none, δ* ← 0, max_FR ← 0;
13     for u ∈ V \ C_s and sum + (s − c(u)) ≤ b do
14         U_V(u) ← UpperBound(G', u, s);
15         L.append(u);
16     sort(L); // by upper bound decreasing order
17     for u ∈ L do
18         if max_FR > 2 · U_V(u) then
19             break;
20         for v ∈ L s.t. v appears after u do
21             if u.component = v.component then
22                 continue;
23             if max_FR > U_V(u) + U_V(v) then
24                 break;
25             if max_FR > U_E(u, v) then
26                 continue;
27             e ← (u, v);
28             δ̃_e ← s − min{c(u), c(v)};
29             if sum + δ̃_e ≤ b then
30                 δ̄_e ← s − w_{G'}(e);
31                 δ_test ← min{δ̄_e, b − sum};
32                 F(e, δ_test) ← Followers(G', e, δ_test, s, C_s);
33                 if F(e, δ_test) ≠ ∅ then
34                     H ← C_s ∪ F(e, δ_test);
35                     δ_e ← max{s − w(u, G'[H]), s − w(v, G'[H])};
36                     FR(e) ← |F(e, δ_test)|/δ_e;
37                     if FR(e) > max_FR or (FR(e) = max_FR and
                        e ≺_tb e*) then
38                         e* ← e, δ* ← δ_e;
39                         max_FR ← FR(e);
40     if e* = none then
41         break;
42     w_{G'}(e*) ← w_{G'}(e*) + δ*;
43     sum ← sum + δ*;
44     A ← A ∪ {⟨e*, δ*⟩};
45     updateComponents(G',b,s,sum,M);
46 return A
```

far. At each iteration, GPA selects the current best candidate across all components. It then explores candidate edges whose endpoints belong to different components, guided by node-level and edge-level upper bounds that allow early termination and safe pruning. The routine searchComponent(G', c, r) returns the best feasible candidate inside component c under the remaining budget r: it runs the same pruned scan as Lines 17–39 restricted to the nodes of c (including, for each $u \in c$, the representative edge from u to the s-core of Theorem 6), and returns no entry if no candidate has $\delta_e \le r$. buildComponents applies it to every initial component, and updateComponents applies it to the components affected by the selected action. If the minimum-identifier node in the s-core changes, updateComponents re-searches all components to preserve lexicographic tie-breaking. Only candidate pairs that cannot be ruled out by these bounds are evaluated using the same follower computation

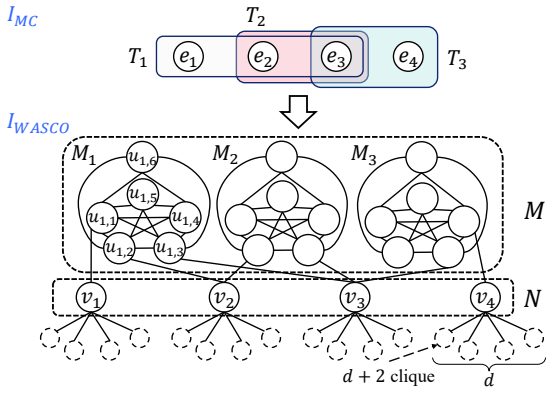


Figure 1: Construction for the NP-hardness proof.

and MEI calculation as in GIA. After applying the selected edge–increment action, GPA updates the affected components and their best candidates while reusing results from unchanged components when feasible. If the best cached candidate exceeds the remaining budget, its component is re-searched under the current budget *before* the cached entry is used as the incumbent, so that *max_FR* always refers to a feasible candidate and the pruning rules never discard a feasible pair on the basis of an infeasible ratio. Ties in follower ratio, both among cached entries and among newly evaluated pairs, are broken by $\prec_{tb}$. The s-core is updated locally within the affected regions. This process repeats until the budget is exhausted or no feasible positive-gain candidate remains, yielding the same greedy outcome as GIA with fewer candidate evaluations.

D Proof of Theorem 1

PROOF. We reduce from the Maximum Coverage (MC) problem, which is NP-hard [1]. Given a universe $U = \{e_1, \dots, e_d\}$, a collection of sets $T_1, \dots, T_t$ with $\bigcup_{i=1}^{t} T_i = U$, and a budget b', MC maximises the number of elements covered by at most b' sets.

We restrict attention to instances satisfying $d \ge 3$, $1 \le b' \le t$, and $T_i \subsetneq U$ for every i. These restrictions preserve NP-hardness: instances with a constant-size universe, zero budget, a budget exceeding the number of sets, or a set equal to U are solvable in polynomial time. Since coverage is monotone, an optimal solution can be extended to select exactly b' sets.

For such an instance, construct a WASCO instance (G, s, b) as follows, as illustrated in Figure 1.

- The graph has three parts: set gadgets M_i, element nodes $N = \{v_1, \dots, v_d\}$, and supporting cliques.
- For each T_i, create $M_i = \{u_{i,1}, \dots, u_{i,d+2}\}$. The nodes $\{u_{i,1}, \dots, u_{i,d+1}\}$ form a clique, and $u_{i,d+2}$ is adjacent to $\{u_{i,1}, \dots, u_{i,d}\}$. Thus, $G[M_i]$ is a $(d+2)$-clique missing only the edge $(u_{i,d+1}, u_{i,d+2})$.
- For every $e_j \in T_i$, add the edge $(u_{i,j}, v_j)$.
- For each v_j, create d fresh $(d+2)$-cliques and connect v_j to one node in each clique. All supporting cliques are mutually node-disjoint and disjoint from the set gadgets and element nodes.
- Set $s = d + 1$ and $b = b'$. Every initial edge has weight one.

The construction takes polynomial time. Let C be the set of all supporting-clique nodes. Every node in C has at least $d+1 = s$ neighbours within its clique, so $C \subseteq C_s(G)$. In each gadget, $u_{i,d+1}$ and $u_{i,d+2}$ initially have degree $d < s$ and are removed. Each remaining gadget node then has at most d neighbours: $d-1$ within its gadget and at most one element node. Consequently, all gadget nodes are removed. Each element node is then left with exactly d supporting-clique neighbours and is also removed. Therefore, $C_s(G) = C$.

Call an intervention *canonical* if it selects b distinct gadgets and adds one unit of weight to the missing edge $(u_{i,d+1}, u_{i,d+2})$ of each selected gadget. For an index set S of selected gadgets, every selected gadget becomes a $(d+2)$-clique and survives. An element node survives if and only if its element belongs to $\bigcup_{i\in S} T_i$: it has d neighbours in C and needs one additional surviving gadget neighbour. Unselected gadgets still collapse as above. Hence, the follower gain of this canonical intervention is

$$b(d+2) + \left|\bigcup_{i\in S} T_i\right|$$

We next show that every feasible intervention can be replaced by a canonical intervention with no smaller gain. Consider an arbitrary feasible intervention, and let x_{uv} be its total increment on the unordered pair $\{u, v\}$. Thus,

$$\sum_{\{u,v\}\subseteq V} x_{uv} \le b$$

Let K be the resulting s-core. For each $u \in K$, define its additional support within K as

$$\eta(u) = \sum_{v\in K\setminus\{u\}} x_{uv}$$

Each unit of edge-weight increment contributes at most two units of additional support within K, so

$$\sum_{u\in K} \eta(u) \le 2b$$

Call M_i *active* if $K \cap M_i \neq \emptyset$. We claim that every active gadget requires at least two units of additional support:

$$\sum_{u\in K\cap M_i} \eta(u) \ge 2$$

To prove this, let $h = |K \cap M_i|$.

- If $1 \le h \le d$, each surviving gadget node has at most $h-1$ neighbours within M_i and at most one element-node neighbour. Its original weighted degree in K is thus at most h, requiring at least $d+1-h$ additional support. Summing over the h surviving nodes gives
$$\sum_{u\in K\cap M_i} \eta(u) \ge h(d+1-h) \ge d \ge 2$$
- If $h = d+1$, exactly one gadget node is absent. If it is an ordinary node $u_{i,j}$ with $j \le d$, both $u_{i,d+1}$ and $u_{i,d+2}$ have original degree $d-1$ in K and each need two additional units. Otherwise, one of these two nodes is absent, and the other has degree d and needs one unit. Since $T_i \subsetneq U$, at least one ordinary node has no element-node neighbour and also needs one unit.
- If $h = d+2$, all gadget nodes survive. Both $u_{i,d+1}$ and $u_{i,d+2}$ have original degree d in K and each need one additional unit.

Let I index the active gadgets, let $k = |I|$, and write $U_I = \bigcup_{i\in I} T_i$. Let q count the surviving element nodes whose elements are outside U_I. Each such element node has no original neighbour in a surviving gadget and has only its d supporting-clique neighbours. It therefore requires at least one unit of additional support. The active gadget nodes and these q element nodes are disjoint, so

$$2k + q \le \sum_{u\in K} \eta(u) \le 2b$$

In particular, $k \le b$.

Choose any index set $S \supseteq I$ with $|S| = b$, which exists because $b \le t$, and apply its canonical intervention. The original intervention gains at most

$$|K \setminus C| \le k(d+2) + |U_I| + q$$

Since $q \le 2(b-k) \le (d+2)(b-k)$, we obtain

$$\begin{aligned} |K \setminus C| &\le k(d+2) + |U_I| + q \\ &\le b(d+2) + |U_I| \\ &\le b(d+2) + \left|\bigcup_{i\in S} T_i\right| \end{aligned}$$

The last expression is exactly the gain of the canonical intervention for S. Thus, an optimal canonical intervention exists.

The optimal follower gain is therefore $b(d+2)$ plus the optimal MC coverage. Equivalently, subtracting the known constant $|C| + b(d+2)$ from the optimal final s-core size recovers the optimal MC coverage. This polynomial-time reduction proves that WASCO is NP-hard even when all initial edge weights are one. □

E Proof of Theorem 2

PROOF. Suppose that f is submodular. Then, for any two sets A and B of edge–weight increment pairs, $f(A) + f(B) \ge f(A \cup B) + f(A \cap B)$. Fix any $s \ge 2$ and consider a graph on four nodes $\{u, v, x, y\}$ with two edges (u, x) and (v, y), each of weight $s-1$. Every node has weighted degree $s-1 < s$, so $C_s = \emptyset$. Let $A = \{\langle (u, v), 1\rangle\}$ and $B = \{\langle (x, y), 1\rangle\}$. Applying A alone raises the weighted degrees of u and v to s, but x and y remain at $s-1$ and are peeled; their removal lowers u and v to $1 < s$, so they are peeled as well. Hence $f(A) = 0$, and by symmetry $f(B) = 0$. Applying $A \cup B$ creates the cycle u–x–y–v–u with edge weights $s-1, 1, s-1, 1$, in which every node has weighted degree exactly s; thus all four nodes enter the s-core and $f(A \cup B) = 4$. Since $f(A \cap B) = f(\emptyset) = 0$, we obtain $f(A) + f(B) = 0 < 4 = f(A \cup B) + f(A \cap B)$, a contradiction. □

F Proof of Theorem 5

PROOF. We first show that the proposed increment is sufficient and minimal, and then establish follower stability. Throughout, we use the monotonicity of the s-core with respect to edge weights: increasing the weight of an edge can only enlarge the s-core.

Sufficiency. Let $\widehat{\delta} = \max\{s - w(u, G[H]), s - w(v, G[H])\}$. Since H is the s-core obtained by anchoring e with $\overline{\delta}_e$, every node in H satisfies the s-core constraint, and $u, v \in H$ as shown in the proof of Theorem 4. We now consider the same node set under the original weights in $G[H]$. As only the weight of $e = (u, v)$ changes, every node in $H \setminus \{u, v\}$ still satisfies the constraint. Hence, H satisfies the

s-core constraint under an increment δ if and only if

$$\delta \geq s - w(u, G[H]), \qquad \delta \geq s - w(v, G[H])$$

Note that $\widehat{\delta} \geq 1$; otherwise, H would satisfy the s-core constraint in G, so $H \subseteq C_s(G)$, contradicting that an endpoint outside $C_s(G)$ belongs to H. By its definition, $\widehat{\delta}$ satisfies both inequalities. Thus, H satisfies the s-core constraint under $\widehat{\delta}$, giving $H \subseteq C_s(G_{e,\widehat{\delta}})$.

Since $\overline{\delta}_e$ provides sufficient weighted degree for both endpoints in H, their required increments cannot exceed $\overline{\delta}_e$. Hence, $\widehat{\delta} \leq \overline{\delta}_e$. By monotonicity, $C_s(G_{e,\widehat{\delta}}) \subseteq H$. Hence, $C_s(G_{e,\widehat{\delta}}) = H$. Since $C_s(G) \subseteq H$ and at least one endpoint was originally outside $C_s(G)$ but belongs to H, this increment induces the non-empty follower set $H \setminus C_s(G)$.
Minimality. Suppose that a smaller positive increment $\delta < \widehat{\delta}$ induces followers, and let $K = C_s(G_{e,\delta})$. By monotonicity of the s-core, $K \subseteq H$. Moreover, as shown in the proof of Theorem 4, both endpoints must belong to K.

By the definition of $\widehat{\delta}$, there exists an endpoint $x \in \{u, v\}$ such that $\widehat{\delta} = s - w(x, G[H])$. Since $\delta < \widehat{\delta}$ and $K \subseteq H$, the weighted degree of x in $G_{e,\delta}[K]$ satisfies

$$w(x, G[K]) + \delta \leq w(x, G[H]) + \delta < s.$$

This contradicts $x \in K$, which requires a weighted degree of at least s. Thus, no smaller increment can induce followers, i.e., $\mathcal{F}(e, \delta) = \emptyset$ for all $\delta < \widehat{\delta}$, proving $\delta_e = \widehat{\delta}$.
Follower stability. Finally, consider any increment $\delta > \delta_e$, and let $K' = C_s(G_{e,\delta})$. Since $C_s(G_{e,\delta_e}) = H$ by sufficiency, monotonicity gives $H \subseteq K'$. We show that K' also satisfies the s-core constraint when the increment is reduced to δ_e. Every node in $K' \setminus \{u, v\}$ retains its weighted degree, since only the weight of e changes. Both endpoints also satisfy the constraint, as they already have sufficient weighted degree within $H \subseteq K'$ under δ_e. Thus, K' satisfies the s-core constraint under δ_e, implying

$$K' \subseteq C_s(G_{e,\delta_e}) = H$$

Together with $H \subseteq K'$, this gives $K' = H$. Therefore, any increment $\delta \geq \delta_e$ induces the same follower set $H \setminus C_s(G)$. □

G Proof of Property 1

Ties in follower ratio are broken by a fixed total order $\prec_{\text{tb}}$, the lexicographic order of the canonical pairs $(\min\{u, v\}, \max\{u, v\})$. Both GIA and GPA rank candidates by the *greedy order* $\triangleright$: $e \triangleright e'$ if $FR(e) > FR(e')$, or $FR(e) = FR(e')$ and $e \prec_{\text{tb}} e'$. As follower ratios are compared exactly, $\triangleright$ is a strict total order: every finite candidate set has a unique $\triangleright$-maximum, regardless of scan order. For the current graph G_t and remaining budget r_t, the *feasible* candidates are

$$\mathcal{E}_t = \{e = (u, v) : u \neq v,\ \{u, v\} \not\subseteq C_s(G_t),\ \delta_e \leq r_t\}, \qquad (1)$$

where δ_e is the MEI of e in G_t.
Candidate evaluation. Both algorithms evaluate candidates by the procedure of Section V-B. By Theorems 4 and 5, it discards e if and only if $\delta_e > r_t$, and otherwise returns exactly δ_e and $\mathcal{F}(e, \delta_e)$. Although $\delta_{\text{test}}(e)$ depends on r_t, the returned values do not: the budget matters only through $\delta_e \leq r_t$, and δ_e, $\mathcal{F}(e, \delta_e)$, and $FR(e)$ depend on G_t alone. GIA selects the $\triangleright$-maximum of $\mathcal{E}_t$ and terminates when $\mathcal{E}_t = \emptyset$, possibly with $r_t > 0$.

PROOF. We show by induction on t that GPA and GIA enter iteration t in the same state (G_t, r_t) and then either both terminate or both select the same edge–increment pair.
Base case. Both start with $(G_1, r_1) = (G, b)$, hence the same $C_s(G)$, coreness–layer pairs, $\tilde{\delta}_u$, $P(u)$, and $U_V(u)$.
Inductive step. Assume both algorithms selected the same edge–increment pairs in the first $t - 1$ iterations; then they share G_t, r_t, $C_s(G_t)$, the coreness–layer pairs, and $\mathcal{E}_t$. Let e^* be the $\triangleright$-maximum of $\mathcal{E}_t$, which GIA selects. To show that GPA also selects e^*, we establish two properties: (i) every candidate retained by GPA for selection is feasible and has the exact MEI and follower ratio; and (ii) GPA never discards e^*.

We prove these properties through candidate reduction, upper-bound pruning, and intermediate-result reuse (as shown in Items 1–3 below). Let ρ be the follower ratio of the best candidate held by GPA during iteration t ($\rho = 0$ initially). Since every retained candidate is feasible, $\rho \leq FR(e^*)$ whenever $\mathcal{E}_t \neq \emptyset$.

(1) **Candidate reduction.** GPA considers only nodes $u \notin C_s(G_t)$ with $\tilde{\delta}_u \leq r_t$; otherwise every candidate e at u has $\delta_e \geq \tilde{\delta}_e \geq \tilde{\delta}_u > r_t$ by Theorem 4, so $e \notin \mathcal{E}_t$. If both endpoints of e^* lie outside $C_s(G_t)$, it is unaffected; otherwise let $e^* = (u, v)$ with $v \in C_s(G_t)$. By Theorem 6, all candidates in $\mathcal{T}_u = \{(u, x) : x \in C_s(G_t)\}$ share the MEI and follower set of e^*, hence its follower ratio and feasibility; as the $\triangleright$-maximum of $\mathcal{E}_t \supseteq \mathcal{T}_u$, e^* is the $\prec_{\text{tb}}$-minimal member of $\mathcal{T}_u$. This is the representative (u, z_t) kept by GPA, with z_t the smallest identifier in $C_s(G_t)$, since $(u, x) \prec_{\text{tb}} (u, x')$ whenever $x < x'$. At each iteration, GPA sets z_t to the smallest identifier in $C_s(G_t)$ to preserve tie-breaking.
(2) **Upper-bound pruning.** By Theorems 3 and 4 and Definition 13, every feasible candidate $e = (u, v)$ with both endpoints outside $C_s(G_t)$ satisfies $FR(e) \leq U_E(u, v) \leq U_V(u) + U_V(v)$. For a representative $e = (u, z_t)$, $FR(e) \leq U_V(u)$.
GPA discards a candidate only if ρ *strictly* exceeds its follower-ratio upper bound: $U_E(u, v)$ for candidate skipping; $U_V(u) + U_V(v)$ for partner-level termination, valid for every later partner v' since partners are scanned in descending $U_V(\cdot)$ ($U_E(u, v') \leq U_V(u) + U_V(v') \leq U_V(u) + U_V(v)$); and $2 \cdot U_V(u)$ for global termination, since later nodes have bounds at most $U_V(u)$. If e^* were discarded with bound B, then $FR(e^*) \leq B < \rho \leq FR(e^*)$, a contradiction. Strictness keeps candidates that may tie ρ and win under $\prec_{\text{tb}}$; the scan order only affects how fast ρ grows. The same holds for the pruned search within a region (item 3).
(3) **Intermediate-result reuse.** GPA maintains regions of non-s-core nodes, initially the connected components of $G[V \setminus C_s(G)]$. After each selection, the regions containing an endpoint of the selected edge are merged and re-evaluated. Thus, every connected component of $G_t[V \setminus C_s(G_t)]$ remains within a region. For each region R, let Ω_R contain its internal candidates and representatives (u, z_t), and let c_R be the $\triangleright$-maximum cached edge when R was last searched at iteration $t' \leq t$. All other candidates are inter-region.
Locality. By Theorem 3, followers of a selected edge remain within the regions containing its non-s-core endpoints. Thus, an unaffected region R retains its internal structure and connections to the s-core; otherwise, a newly engaged neighbour would have belonged to the same non-s-core component. By Theorem 5, the

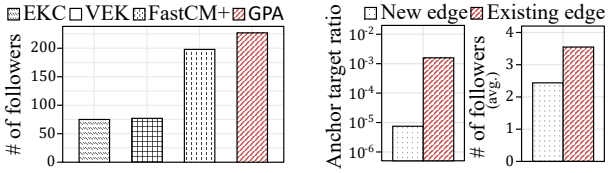
MEI, follower set, and follower ratio of every candidate in Ω_R remain unchanged. If z_t changes, GPA re-evaluates all regions to preserve tie-breaking.

Shrinking budget. Since $r_t \le r_{t'}$, some cached candidates may become infeasible. If $\delta_{c_R} \le r_t$, the cached c_R remains the best feasible candidate in R and can be reused. Otherwise, R must be re-searched to find its current best candidate. However, this search can be deferred if another feasible cached candidate $c \triangleright c_R$ exists, since c_R still outranks every candidate in R. Thus, deferring the search cannot exclude the globally best feasible candidate from consideration.

Preservation of e^*. If $e^* \in \Omega_R$, it is either the valid cached maximum or is found by re-searching R. In the latter case, re-evaluation cannot be deferred, since no feasible candidate outranks e^*. Otherwise, e^* is inter-region and is evaluated afresh; by Item 2, it cannot be pruned. Thus, e^* reaches the final comparison under $\triangleright$, establishing completeness.

If $\mathcal{E}_t = \varnothing$, both algorithms terminate. Otherwise, Items 1–3 establish that GPA retains e^* with its exact MEI and follower ratio, even when retrieved from the cache. Thus, both algorithms select the same edge–increment pair, yielding identical G_{t+1} and $r_{t+1} = r_t - \delta_{e^*}$. By Definition 5 and the locality established above, GPA's local s-core update agrees with GIA's global recomputation. Hence, by induction, both algorithms select identical edge–increment pairs, terminate together, and return the same final s-core. □

H Additional Experiment



(a) $k = 20, b = 200$ **(b) Anchored edge analysis**

Figure 2: Effectiveness on the Facebook dataset

Effectiveness (unweighted graph). We further evaluate GPA on the unweighted Facebook dataset [2] ($|V| = 4,039$, $|E| = 88,234$, $d_{\text{avg}} = 43.7$). The baseline methods, including EKC [4], VEK [5], and FastCM+ [3], are originally designed for unweighted graphs. To enable a fair comparison, we treat the unweighted graph as a special case of our weighted formulation by assigning unit weight to every edge, and follow the same parameter settings ($k = 20$, $b = 200$) as in FastCM+. As shown in Figure 2(a), GPA can achieve the largest number of followers among all compared methods under the same budget. This result indicates that, even when restricted to the unweighted setting, GPA is capable of operating effectively in practice. Figure 2(b) further analyses the anchoring behaviour of GPA. After normalising by the size of the corresponding feasible action spaces, edge reinforcement is selected more frequently than new edge insertion. Moreover, reinforcement actions tend to yield a higher average number of followers per anchored edge. These observations suggest that strengthening existing connections can be a more effective anchoring strategy than edge insertion, even when all edges have uniform weight.

References

[1] Samir Khuller, Anna Moss, and Joseph Seffi Naor. 1999. The budgeted maximum coverage problem. *Information processing letters* 70, 1 (1999), 39–45.

[2] Jure Leskovec and Julian Mcauley. 2012. Learning to discover social circles in ego networks. *Advances in neural information processing systems* 25 (2012).

[3] Xin Sun, Xin Huang, and Di Jin. 2022. Fast algorithms for core maximization on large graphs. *Proceedings of the Very Large Data Bases (VLDB) Endowment* 15, 7 (2022), 1350–1362.

[4] Zhongxin Zhou, Fan Zhang, Xuemin Lin, Wenjie Zhang, and Chen Chen. 2019. k-Core Maximization: An Edge Addition Approach.. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 4867–4873.

[5] Zhongxin Zhou, Wenchao Zhang, Fan Zhang, Deming Chu, and Binghao Li. 2022. VEK: a vertex-oriented approach for edge k-core problem. *World Wide Web* 25, 2 (2022), 723–740.